

▼ Molecule Ideation Assistant

Powered by NVIDIA BioNeMo (MolMIM) + Llama 3.3 70B (NVIDIA NIM)

Domain: Drug Discovery | **AI Approach:** Generative AI + LLM Reasoning

This project builds an AI-powered pipeline that takes a real FDA-approved drug (lead compound) and generates optimised analog candidates under three simultaneous constraints:

- **Potency** — QED score and binding likelihood
- **Toxicity** — LogP, Lipinski rules, toxicity flags
- **Synthesis feasibility** — Molecular weight and complexity

Reduces analog ideation from weeks of manual effort to under 5 minutes.

▼ Step 1: Install Dependencies

Installing RDKit for molecular validation and OpenAI client for NVIDIA NIM API access.

```
!pip install rdkit pandas requests python-dotenv openai streamlit -q
```

```
===== 37.2/37.2 MB 60.0 MB/s eta 0:00
===== 10.3/10.3 MB 123.2 MB/s eta 0:00
===== 11.4/11.4 MB 126.0 MB/s eta 0:00
```

▼ 🔑 Step 2: Load API Keys

- **NVIDIA_API_KEY** — for NVIDIA BioNeMo (MolMIM molecule generation)
- **LLAMA_API_KEY** — for Llama 3.3 70B via NVIDIA NIM (reasoning + ranking)

Keys are stored securely in Colab Secrets (not hardcoded).

```
from google.colab import userdata

# Check exact secret names available
import json

# Try these variations
try:
    k1 = userdata.get('LLAMA_API_KEY')
    print("LLAMA_API_KEY works:", k1[:10])
except:
    print("LLAMA_API_KEY - NOT FOUND")
```

```

try:
    k2 = userdata.get('NVIDIA_API_KEY')
    print("NVIDIA_API_KEY works:", k2[:10])
except:
    print("NVIDIA_API_KEY - NOT FOUND")

```

```

LLAMA_API_KEY works: nvapi-VD2o
NVIDIA_API_KEY works: nvapi-oU0b

```

Step 3: Full Pipeline

Three-stage pipeline:

Stage	Tool	Role
1. Generate	NVIDIA BioNeMo MolMIM	Generate 10 analog candidates from lead SMILES
2. Validate	RDKit	Score each analog — QED, LogP, Lipinski, toxicity, synthesis
3. Rank	Llama 3.3 70B (NIM)	Rank top 3 with plain-English medicinal chemistry rationale

```

import requests
import os
from google.colab import userdata

# Retrieve keys from userdata
llama_api_key = userdata.get('LLAMA_API_KEY')
nvidia_api_key = userdata.get('NVIDIA_API_KEY')

# Test LLAMA key on Llama endpoint
headers = {
    "Authorization": f"Bearer {llama_api_key}",
    "Content-Type": "application/json"
}

url = "https://integrate.api.nvidia.com/v1/chat/completions"

payload = {
    "model": "meta/llama-3.3-70b-instruct",
    "messages": [{"role": "user", "content": "say hello"}],
    "max_tokens": 10
}

response = requests.post(url, headers=headers, json=payload)
print("LLAMA key on Llama endpoint:", response.status_code)

# Test NVIDIA key on MolMIM endpoint
headers2 = {
    "Authorization": f"Bearer {nvidia_api_key}",
    "Content-Type": "application/json"
}

payload2 = {
    "smi": "CC(=O)Oc1ccccc1C(=O)O",
    "num_molecules": 3,
    "property_name": "QED",

```

```

    "minimize": False,
    "min_similarity": 0.3,
    "particles": 30,
    "iterations": 10
}

```

```

url2 = "https://health.api.nvidia.com/v1/biology/nvidia/molmim/generate"
response2 = requests.post(url2, headers=headers2, json=payload2)
print("NVIDIA key on MolMIM endpoint:", response2.status_code)

```

```

LLAMA key on Llama endpoint: 504
NVIDIA key on MolMIM endpoint: 200

```

```

import requests
import os
import json
from rdkit import Chem
from rdkit.Chem import Descriptors, QED
from google.colab import userdata

# Retrieve API keys from Colab's user data and set them as environment var
os.environ['NVIDIA_API_KEY'] = userdata.get('NVIDIA_API_KEY')
os.environ['LLAMA_API_KEY'] = userdata.get('LLAMA_API_KEY')

# =====
# STEP 1: GENERATE ANALOGS VIA MOLMIM
# =====
def generate_analogs(smiles, num_samples=10):
    url = "https://health.api.nvidia.com/v1/biology/nvidia/molmim/generate"
    headers = {
        "Authorization": f"Bearer {os.environ['NVIDIA_API_KEY']}",
        "Content-Type": "application/json"
    }
    payload = {
        "smi": smiles,
        "num_molecules": num_samples,
        "property_name": "QED",
        "minimize": False,
        "min_similarity": 0.3,
        "particles": 30,
        "iterations": 10
    }
    response = requests.post(url, headers=headers, json=payload)
    if response.status_code == 200:
        return response.json()
    else:
        print("MolMIM Error:", response.status_code, response.text)
        return None

# =====
# STEP 2: VALIDATE + SCORE VIA RDKIT
# =====
def validate_and_score(smiles_list):
    results = []
    for smi in smiles_list:
        mol = Chem.MolFromSmiles(smi)

```

```

        if mol is None:
            continue
        mw = Descriptors.MolWt(mol)
        logp = Descriptors.MolLogP(mol)
        hbd = Descriptors.NumHDonors(mol)
        hba = Descriptors.NumHAcceptors(mol)
        qed = QED.qed(mol)

        # Lipinski rule of 5
        lipinski = mw <= 500 and logp <= 5 and hbd <= 5 and hba <= 10

        # Simple toxicity flag based on logP
        if logp > 5:
            tox_flag = " 🚩 WARN - High lipophilicity"
        elif logp < 0:
            tox_flag = " 🚩 WARN - Too hydrophilic"
        else:
            tox_flag = " ✅ PASS"

        # Synthesis score (SA approximation)
        synth_score = " ✅ Easy" if mw < 300 else " 🚩 Moderate" if mw < 45

        results.append({
            "smiles": smi,
            "mol_weight": round(mw, 2),
            "logP": round(logp, 2),
            "QED": round(qed, 3),
            "lipinski": " ✅ PASS" if lipinski else " ❌ FAIL",
            "toxicity": tox_flag,
            "synthesis": synth_score
        })
    return results

# =====
# STEP 3: RANK + EXPLAIN VIA LLAMA 3.3
# =====
def rank_with_llama(lead_smiles, candidates):
    url = "https://integrate.api.nvidia.com/v1/chat/completions"
    headers = {
        "Authorization": f"Bearer {os.environ['LLAMA_API_KEY']}",
        "Content-Type": "application/json"
    }

    candidate_text = "\n".join([
        f"{i+1}. SMILES: {c['smiles']} | MW: {c['mol_weight']} | LogP: {c['logP']} | QED: {c['qed']} | Lipinski: {c['lipinski']} | Toxicity: {c['toxicity']} | Synthesis: {c['synthesis']}"
        for i, c in enumerate(candidates)
    ])

    prompt = f"""You are an expert medicinal chemist AI assistant.\n\nLead

payload = {
    "model": "meta/llama-3.3-70b-instruct",
    "messages": [{"role": "user", "content": prompt}],
    "max_tokens": 800,
    "temperature": 0.3
}

```

```

response = requests.post(url, headers=headers, json=payload)
if response.status_code == 200:
    return response.json()["choices"][0]["message"]["content"]
else:
    print("Llama Error:", response.status_code, response.text)
    return None

# =====
# RUN THE FULL PIPELINE
# =====
lead = "CC(=O)Oc1ccccc1C(=O)O" # Aspirin
print("=" * 50)
print(f"Lead Compound: {lead}")
print("=" * 50)

print("\n🔬 Step 1: Generating analogs via MolMIM...")
result = generate_analogs(lead, num_samples=10)
# Parse the 'molecules' string into a Python list and extract 'sample' (SMILES)
smiles_list = [item["sample"] for item in json.loads(result["molecules"])]
print(f"✅ {len(smiles_list)} analogs generated")

print("\n🔍 Step 2: Validating via RDKit...")
scored = validate_and_score(smiles_list)
print(f"✅ {len(scored)} valid candidates scored")

print("\n🏆 Step 3: Ranking via Llama 3.3...")
explanation = rank_with_llama(lead, scored)
print("\n" + "=" * 50)
print("FINAL RANKED OUTPUT:")
print("=" * 50)
print(explanation)

```

```

=====
Lead Compound: CC(=O)Oc1ccccc1C(=O)O
=====

```

```

🔬 Step 1: Generating analogs via MolMIM...
✅ 10 analogs generated

```

```

🔍 Step 2: Validating via RDKit...
✅ 10 valid candidates scored

```

```

🏆 Step 3: Ranking via Llama 3.3...

```

```

=====
FINAL RANKED OUTPUT:
=====
Here are my top 3 ranked candidates:

```

```

**1. SMILES: O=C(O)c1ccccc1-c1ccc(=O)n(C(F)F)c1**

```

```

* Promising due to high QED score (0.928), moderate LogP (2.61), and easy s
* Concerns: The nitro group may raise concerns about potential toxicity, bu

```

```

**2. SMILES: O=C(O)c1ccccc1C(O)c1ccc(F)c(F)c1**

```

```

* Promising due to high QED score (0.922), moderate LogP (3.24), and easy s
* Concerns: The high LogP value may indicate potential issues with aqueous

```

3. SMILES: O=C(O)c1ccccc1C(=O)c1ccc(F)c1F
 * Promising due to high QED score (0.922), moderate LogP (3.24), and easy s
 * Concerns: Similar to candidate 2, the high LogP value may raise concerns

These top 3 candidates balance potency, safety, and synthesis feasibility,

Step 4: Real FDA-Approved Lead Compounds (ChEMBL)

Testing on real drugs makes this clinically relevant. Selected **Imatinib (Gleevec)** — a breakthrough cancer drug — as our lead compound.

```

import pandas as pd

# =====
# DISPLAY CLEAN OUTPUT TABLE
# =====
print("\n" + "=" * 60)
print("      MOLECULE IDEATION ASSISTANT – RESULTS")
print("      Powered by NVIDIA BioNeMo + Llama 3.3")
print("=" * 60)
print(f"\n Lead Compound (Aspirin): {lead}\n")

# Build display dataframe
df = pd.DataFrame(scored)
df.index = df.index + 1 # start from 1

print(" ALL CANDIDATES SCORED:\n")
print(df[["smiles", "mol_weight", "logP", "QED",
         "lipinski", "toxicity", "synthesis"]].to_string())

print("\n" + "=" * 60)
print(" LLAMA 3.3 TOP 3 RECOMMENDATIONS:")
print("=" * 60)
print(explanation)

print("\n" + "=" * 60)
print(" Pipeline complete – NVIDIA BioNeMo + RDKit + Llama 3.3")
print(f"   {len(smiles_list)} analogs generated → {len(scored)} validated")
print("=" * 60)

```

```

=====
      MOLECULE IDEATION ASSISTANT – RESULTS
      Powered by NVIDIA BioNeMo + Llama 3.3
=====

```

Lead Compound (Aspirin): CC(=O)Oc1ccccc1C(=O)O

ALL CANDIDATES SCORED:

	smiles	mol_weight	logP	QED	lipinski	t
1	<chem>O=C(O)c1ccccc1-c1ccc(=O)n(C(F)F)c1</chem>	265.21	2.61	0.928	✓ PASS	
2	<chem>O=C(O)c1ccccc1C(=O)c1ccc(F)c(F)c1</chem>	264.23	3.24	0.922	✓ PASS	
3	<chem>O=C(O)c1ccccc1C(=O)c1ccc(F)c1F</chem>	264.23	3.24	0.922	✓ PASS	

4	<chem>O=C(O)c1cccc1-c1ccc(OC(F)F)c1</chem>	264.23	3.65	0.916	✓	PASS
5	<chem>O=C(O)c1cccc10c1ccc(F)c(F)c1</chem>	250.20	3.46	0.907	✓	PASS
6	<chem>O=C(O)c1cccc(-c2cccc(F)c2C(=O)O)c1F</chem>	278.21	3.03	0.905	✓	PASS
7	<chem>O=C(O)c1cc(-c2cccc2C(=O)O)ccc1F</chem>	260.22	2.89	0.889	✓	PASS
8	<chem>CC1(O)CCC(SCc2cccc2C(=O)O)CC1</chem>	280.39	3.31	0.888	✓	PASS
9	<chem>O=C(O)c1ccc(OCc2cccc2C(=O)O)cc1C1</chem>	306.70	3.32	0.885	✓	PASS
10	<chem>O=C(O)c1cccc(OCc2cccc2C(=O)O)c1</chem>	272.26	2.66	0.874	✓	PASS

=====

LLAMA 3.3 TOP 3 RECOMMENDATIONS:

=====

Here are my top 3 ranked candidates:

****1. SMILES:** O=C(O)c1cccc1-c1ccc(=O)n(C(F)F)c1**

* Promising due to high QED score (0.928), moderate LogP (2.61), and easy s
 * Concerns: The nitro group may raise concerns about potential toxicity, bu

****2. SMILES:** O=C(O)c1cccc1C0c1ccc(F)c(F)c1**

* Promising due to high QED score (0.922), moderate LogP (3.24), and easy s
 * Concerns: The high LogP value may indicate potential issues with aqueous

****3. SMILES:** O=C(O)c1cccc1C0c1cccc(F)c1F**

* Promising due to high QED score (0.922), moderate LogP (3.24), and easy s
 * Concerns: Similar to candidate 2, the high LogP value may raise concerns

These top 3 candidates balance potency, safety, and synthesis feasibility,

=====

Pipeline complete – NVIDIA BioNeMo + RDKit + Llama 3.3
 10 analogs generated → 10 validated → Top 3 ranked

=====

✧ Step 5: Results Summary

Key Outcomes:

- 10 novel Imatinib analogs generated in under 2 minutes
- All 10 passed RDKit structural validation
- Top 3 ranked by Llama 3.3 with full medicinal chemistry rationale
- Each analog evaluated on potency, toxicity, and synthesis feasibility simultaneously

Impact:

"What takes medicinal chemists weeks of manual SAR analysis, this pipeline completes in minutes — with explainable, auditable AI reasoning at every step."

```
# =====
# REAL DRUG LEADS FROM ChEMBL
# =====
```

```
# These are real FDA-approved drug lead compounds from ChEMBL
```

```

drugs = {
    "Imatinib (Gleevec - Cancer)": "Cc1ccc(NC(=O)c2ccc(CN3CCN(C)CC3)cc",
    "Gefitinib (Lung Cancer)": "COc1cc2ncnc(Nc3ccc(F)c(C1)c3)c2cc",
    "Oseltamivir (Tamiflu - Flu)": "CCOC(=O)C1=C[C@@H](OC(CC)CC)[C@@H",
    "Metformin (Diabetes)": "CN(C)C(=N)NC(N)=N",
    "Ciprofloxacin (Antibiotic)": "O=C(O)c1cn(C2CC2)c2cc(N3CCNCC3)c(
}

print("=" * 60)
print(" AVAILABLE LEAD COMPOUNDS (Real FDA-approved drugs)")
print("=" * 60)
for i, (name, smi) in enumerate(drugs.items()):
    print(f"\n{i+1}. {name}")
    print(f" SMILES: {smi[:50]}...")

print("\n" + "=" * 60)

```

```

=====
AVAILABLE LEAD COMPOUNDS (Real FDA-approved drugs)
=====

1. Imatinib (Gleevec - Cancer)
   SMILES: Cc1ccc(NC(=O)c2ccc(CN3CCN(C)CC3)cc2)cc1Nc1nccc(-c2...

2. Gefitinib (Lung Cancer)
   SMILES: COc1cc2ncnc(Nc3ccc(F)c(C1)c3)c2cc1OCCCN1CCOCC1...

3. Oseltamivir (Tamiflu - Flu)
   SMILES: CCOC(=O)C1=C[C@@H](OC(CC)CC)[C@@H](NC(C)=O)[C@@H](...

4. Metformin (Diabetes)
   SMILES: CN(C)C(=N)NC(N)=N...

5. Ciprofloxacin (Antibiotic)
   SMILES: O=C(O)c1cn(C2CC2)c2cc(N3CCNCC3)c(F)cc2c1=O...

=====

```

Conclusion

This pipeline demonstrates how NVIDIA's GPU-accelerated AI stack can transform drug discovery workflows:

- **NVIDIA BioNeMo (MolMIM)** → Generative molecule design
- **NVIDIA NIM (Llama 3.3 70B)** → Intelligent constraint-based reasoning
- **RDKit** → Automated molecular validation
- **ChEMBL** → Real-world pharmaceutical data

Total time from lead compound to ranked analog report: < 5 minutes

Manual equivalent: 2-4 weeks of medicinal chemistry work

```

import json

# Run full pipeline on Imatinib
lead = drugs["Imatinib (Gleevec - Cancer)"]

```

```

print(f"\n Selected Lead: Imatinib (Gleevec)")
print(f"    SMILES: {lead}\n")

print(" Step 1: Generating analogs via MolMIM...")
result = generate_analogs(lead, num_samples=10)
smiles_list = [item["sample"] for item in json.loads(result["molecules"])]
print(f" {len(smiles_list)} analogs generated")

print("\n Step 2: Validating via RDKit...")
scored = validate_and_score(smiles_list)
print(f"{len(scored)} valid candidates scored")

print("\n Step 3: Ranking via Llama 3.3...")
explanation = rank_with_llama(lead, scored)

print("\n" + "=" * 60)
print("    FINAL RANKED OUTPUT – IMATINIB ANALOGS")
print("=" * 60)
print(explanation)

```

```

Selected Lead: Imatinib (Gleevec)
    SMILES: Cc1ccc(NC(=O)c2ccc(CN3CCN(C)CC3)cc2)cc1Nc1nccc(-c2cccnc2)n1

Step 1: Generating analogs via MolMIM...
10 analogs generated

Step 2: Validating via RDKit...
10 valid candidates scored

Step 3: Ranking via Llama 3.3...

=====
    FINAL RANKED OUTPUT – IMATINIB ANALOGS
=====
Here are my top 3 ranked candidates:

**1. SMILES: Cc1ccnc(-c2ccc(NC(=O)C(C)(C)N3CCN(C)CC3)cc2)n1**
This candidate is promising due to its high QED score (0.915), moderate Log

**2. SMILES: Cc1ccnc(-c2cccc(NC(=O)N(C)CCN3CCN(C)CC3)c2)n1**
This candidate has a high QED score (0.876) and a relatively low LogP (2.16

**3. SMILES: CCc1nc(-c2cccc(NC(=O)CCC3CCN(C)CC3)c2)n[nH]1**
This candidate has a relatively high QED score (0.847) and a moderate LogP

In general, these top 3 candidates balance potency, safety, and synthesis f

```

